

Execution Realism Layer

SPY Bit-Exact Partial-Parity Validation Report

Run: 2026-04-22T21:09:25+00:00 · Branch: execution-realism-layer (merged to master) · Runtime: 1.57s · Engine window: 2025-4-21 → 2026-3-27

TL;DR. The four-phase execution-realism layer (slippage/commission config, pessimistic intrabar TP/SL resolver, session wrapper, and resting limit orders) was shipped with 52 passing unit tests and an explicit design guarantee that strategies not using the new features would traverse dormant code paths. This report validates that guarantee against real LEAN reference data: **33 of 33 SPY EMA-crossover trades match the committed LEAN fixture bit-exactly** over the 10-month window for which local minute data is available. No divergences remain after correcting a data-shape mismatch (extended-hours bars in the local cache vs. RTH-only data in the reference fixture). The engine's entry/exit timing, price capture, EMA(5), EMA(10), RSI(14), PnL points, and PnL percent all reproduce LEAN's trade log to the last reported decimal.

1 · What This Validates

The goal is not to re-validate LEAN itself; the committed fixture at `app/engine/tests/fixtures/spy_lean_trades.csv` is treated as ground truth. The goal is to confirm that the execution-realism layer shipped in PRs 1–4 did *not* perturb the engine's behavior when run against a strategy that does not opt into any of the new features. The SPY EMA crossover algorithm (`SpyEmaCrossoverAlgorithm`) is an ideal probe because:

- It submits **market** orders only — so the new limit-order resting book in `BacktestEngine` is never touched.
- It never attaches `take_profit_price` or `stop_loss_price` — so the bracket-watcher code path is dormant.
- It does not configure `session_entry_cutoff` or `force_flat_at` — so the session wrapper's cancellation logic is dormant.
- It uses `SIGNAL_BAR_CLOSE` fills with a default \$1/order commission and zero slippage — so the `ExecutionConfig` defaults must produce byte-identical output to a pre-PR-1 `FillModel()` construction.

If any of the new code paths had accidentally activated for this strategy (e.g. a bracket being registered for every market order, or a session filter running with a `None`-valued cutoff), bit-exact trade-by-trade equivalence would break immediately. It does not.

2 · Methodology

Local minute data is available only for SPY from **2025-04-21 to 2026-04-20** — roughly the final year of the fixture's 2024-03-28 → 2026-03-27 window. The fixture contains 63 trades spanning the full LEAN run; 36 fall inside the cached window, and 33 occur at or after the 2-week post-start buffer (2025-05-05) used to let the indicator recursions settle.

2.1 · Why a warmup buffer is required

EMA and RSI are recursive with an initial-seed component (SMA of the first N samples for EMA; SMA of first N gains/losses for Wilders RSI). Running the engine from 2025-04-21 produces different seed values than running from 2024-03-28. The difference decays exponentially at rate $(1-\alpha)^n$, and for EMA(10) with $\alpha = 2/11 \approx 0.182$, initial-seed influence drops below the fixture's 4-decimal display precision after roughly 60 consolidated 15-minute bars (~3 trading days). A 2-week buffer is well above that floor.

2.2 · The partial-parity script

A 170-line diagnostic (`run_spy_partial_parity.py` , not committed) points `LeanMinuteDataReader` at the local cache, overrides the strategy's hardcoded date range to fit the available data, wraps the reader with an RTH filter (see §4), runs the engine, and diffs trade-by-trade against the fixture with the same precision conventions LEAN uses: 2 decimals for prices and PnL points, 4 decimals for EMAs, 2 for RSI, 6 for PnL percent.

3 · Results

Metric	Value
Fixture trades in comparison window	33
Engine trades in comparison window	33
Trade pairings compared	33
Bit-exact matches	33 / 33
Divergences	0
Final equity (engine)	\$105,747.97
Total fees (engine)	\$72.00
Net profit (engine)	\$5,747.97
Engine runtime	1.57s

Every compared field matches to the fixture's reported precision. The specific columns checked on each trade:

Field	Precision	Engine Source
-------	-----------	---------------

entry_time / exit_time	minute (tz-aware America/New_York)	<code>TradeBarConsolidator.end_time</code> → <code>LoggedTrade.entry_time</code>
entry_price / exit_price	2 decimals	<code>FillModel.fill_market_order</code> → <code>OrderEvent.fill_price</code>
ema5 / ema10	4 decimals	<code>ExponentialMovingAverage.current</code> at signal bar
rsi	2 decimals	<code>RelativeStrengthIndex.current</code> (Wilders)
pnl_pts	2 decimals	<code>LoggedTrade.pnl_pts</code> (exit – entry, Decimal)
pnl_pct	6 decimals	<code>LoggedTrade.pnl_pct</code>
result	WIN / LOSS	<code>LoggedTrade.result</code>

4 · The First-Pass Divergence That Almost Was

The first run produced **33 mismatches on 33 pairings** — not a single trade matched. The engine fired its first in-window trade at `2025-05-06 18:15 ET`, but the fixture's first in-window entry was `2025-05-07 15:45 ET`. 18:15 ET is after-hours.

Root cause: the locally-cached SPY zips were fetched from Polygon with extended hours enabled (832 minute bars/day, 04:00 → 19:59 ET). The committed LEAN fixture was produced from RTH-only reference data (390 bars/day, 09:30 → 16:00 ET). The engine happily consolidated extended-hours bars into 15-minute periods, generating signal bars and trades LEAN never saw.

This is a *data-shape* divergence, not an engine bug. Post-hoc filtering of trades to RTH timestamps would not fix it — the EMA and RSI values at any RTH timestamp were built from recursions that had mixed in extended-hours bars, so even the "good" times would produce different indicator values than the fixture.

The fix was a 10-line `RTHFilteredReader` wrapper around `LeanMinuteDataReader` that drops bars whose local time falls outside `[09:30, 16:00)` ET before the consolidator sees them. With the filter active, indicator recursions are fed the same bar set the fixture was built from, and all 33 trades match.

Before RTH filter Engine trades (window): 35 Fixture trades (window): 33 Bit-exact matches: 0 / 33 Final equity: \$102,888.48	After RTH filter Engine trades (window): 33 Fixture trades (window): 33 Bit-exact matches: 33 / 33 Final equity: \$105,747.97
--	--

5 · Code Paths Validated vs Dormant

5.1 • Validated bit-exactly

Each SPY trade round-trips through the following engine code. Match on every field, every trade, confirms the module is behaviorally unchanged after the PR 1–4 changes:

Module	Function(s) exercised
app/engine/data/lean_format.py	<code>LeanMinuteDataReader.iter_bars</code> — zip decode, millisecond-offset parsing, Decimal price scaling, <code>bar.end_time</code> assignment
app/engine/consolidators/trade_bar_consolidator.py	<code>update</code> , <code>_emit_working</code> — epoch-aligned 15-minute rounding, OHLCV aggregation, left-closed interval semantics
app/engine/indicators/*	EMA(5), EMA(10) with SMA-seeded warmup; Wilders RSI(14) with period+1 warmup and smoothed-recursion thereafter
app/engine/strategy/base.py	<code>StrategyContext._on_emit</code> (including the new <code>_pre_handler_hook</code> plumbing from PR 2), consolidator registration, <code>current_time</code> propagation
app/engine/execution/fill_model.py	<code>fill_market_order</code> in <code>SIGNAL_BAR_CLOSE</code> mode, commission application
app/engine/execution/portfolio.py	<code>apply_fill</code> (cash accounting, average price, position flip logic); <code>submit_market_order</code>
app/engine/engine.py	Main loop: reference-price updates, drain step, equity snapshots, LoggedTrade assembly
app/engine/execution/execution_config.py	<code>ExecutionConfig()</code> defaults — proven equivalent to the pre-PR-1 zero-arg <code>FillModel()</code> , since the strategy supplies no overrides

5.2 • Dormant during this run (validated by separate unit tests)

The SPY strategy does not exercise these code paths. Their bit-exact behavior is guaranteed by the 50 unit tests added in PRs 1–4, not by this parity check:

Code path	Test file providing coverage
-----------	------------------------------

Slippage & non-default commission threading	<code>test_execution_config.py</code> (6 tests)
Pessimistic intrabar TP/SL resolver	<code>test_intrabar_resolver.py</code> (14 tests, pure-function)
Engine-managed bracket watcher	<code>test_bracket_exits.py</code> (5 tests, synthetic bars)
Session entry cutoff & force-flat	<code>test_session_wrapper.py</code> (12 tests, including NEXT_BAR_OPEN orphan-fill cancellation and bracket clearing)
Resting limit orders with penetration rule	<code>test_limit_orders.py</code> (13 tests)

6 · Differences and Root Causes

After the RTH-filter correction, no field-level differences remain. The only "difference" surfaced by this exercise was the extended-hours data-shape issue, rooted not in code but in how the local cache was populated. Its proximate cause is likely `PythonDataService/app/engine/data/polygon_export.py`, which fetches Polygon aggregates without filtering to regular-session timestamps. The fixture's LEAN source almost certainly pulled from a QuantConnect reference mount that applies RTH-only pre-processing at the data-provider level.

This is documented in `docs/tv-polygon-validation-gotchas.md` and has surfaced before in unrelated validation work; it is not a new or surprising failure mode. The fix belongs in the cache-population pipeline, not in the engine.

7 · What the 33-Match Result Does *Not* Prove

Clear-eyed scope statement, because positive parity is easy to over-interpret:

- **Trades 1–24 of the fixture** (2024-04-11 → 2025-03-17) are not covered. Local data doesn't reach back that far. An indicator seeding bug that affects only the first ~30 bars of a backtest would not be caught by this test.
- **NEXT_BAR_OPEN fill mode** is not exercised. The separate `test_spy_next_bar_open_validation.py` covers that mode against a committed engine-side baseline, but was not run here because its baseline CSV depends on the same missing pre-2025-04 data.
- **New code paths themselves** are only validated by the unit tests cited in §5.2. This report proves those paths are correctly *off-by-default*, not that they produce correct output when activated by real strategies.

8 · Next Steps

In rough priority order:

1. **Fetch the missing 2024-03-28 → 2025-04-20 SPY range via Polygon export.** The `POST /api/engine/export-lean` endpoint already exists and `POLYGON_API_KEY` is configured. Expected runtime ~5–10 minutes. Re-running the parity script against the full range would yield a 63/63 bit-exact comparison and eliminate the "24 trades not covered" caveat.
2. **Add RTH filtering to the Polygon-to-LEAN export path.** The 10-line `RTHFilteredReader` used here is a workaround; the canonical fix is to drop extended-hours bars during `polygon_export.py`'s write step so every downstream consumer (not just this parity test) sees RTH-shaped data without needing to know.
3. **Exercise the new code paths against a live strategy.** Pick one existing strategy (ORB or RSI mean-reversion are natural candidates because they already have defined exit rules), attach pessimistic TP/SL brackets, and compare the resulting trade log to the unbracketed version. This turns the realism layer from "shipped infrastructure" into "measured reduction in win-rate bias."
4. **Wire the new `EngineBacktestRequest` fields into the Angular Engine Lab form.** Slippage, session cutoff, force-flat, and limit penetration are exposed on the API but not in the UI; users cannot flip them without hitting Swagger directly.
5. **(Deferred, as previously agreed) PR 5: bar magnifier.** 1-minute intrabar replay for TP/SL ambiguity. Only worth building if the pessimistic resolver proves too harsh on strategies we believe in. The partial-parity pass strengthens the case for leaving this deferred — nothing in the engine layer is blocked by its absence.

9 · Reproducibility

```
cd PythonDataService
python3 run_spy_partial_parity.py
# writes _spy_partial_parity_results.json and prints match summary

python3 generate_validation_report.py
# reads the JSON and writes docs/execution-realism-validation-report.pdf
```

The validation script pins:

- Data root: `PythonDataService/lean-cache/` (committed as part of previous fetches)
- Strategy: `SpyEmaCrossoverAlgorithm` with default parameters (SPY, EMA5/10, Wilders RSI14, 15-min bars, \$1 commission, 0 slippage)
- Fill mode: `SIGNAL_BAR_CLOSE`
- Engine window: 2025-04-21 → 2026-03-27
- Comparison window: entries on or after 2025-05-05 (2-week EMA/RSI warmup buffer)